

ARCHITECTURE REVIEW BOARD · IMPLEMENTATION
BLUEPRINT

KIRANA

Agent Financial Control Plane

Commoditization audit, frozen architecture, and build-ready blueprint for delegated agentic payments on India's UPI rails. What survives the kill-list: verifiable delegation, deterministic policy, cryptographic transaction binding — and a proof bundle for every rupee an agent spends.

Office of the Principal Architect

Frozen v1.0 · build-ready · TypeScript / Node · reviewed against the KIRANA research corpus

31 AUGUST 2026

Table of Contents

1. Executive Verdict	1
2. Evidence Base and Method	2
3. The Commoditization Kill-List	3
4. What Survives: The Defensible Wedge	5
5. Frozen Architecture	8
6. Delegation Model and Policy Engine	11
7. Data Model	13
8. API Contracts and State Machines	14
9. Security Model and Threat Matrix	17
10. 90-Day Execution Plan	19
11. Regulatory Boundary Map	20
12. Risk Register and Final Recommendation	22

1. Executive Verdict

This blueprint answers one question: after auditing everything the payments ecosystem already ships, what exactly can KIRANA build that is not already commoditized? The answer, stated bluntly at the outset, is that between 80 and 90 percent of the scope originally imagined for KIRANA is already someone else's product, someone else's licensed activity, or someone else's active roadmap. Payment rails belong to NPCI and the card schemes. Acquiring and orchestration belong to Razorpay, Cashfree, Juspay, and Pine Labs. Mandates belong to UPI AutoPay. Reserved-balance mechanics belong to UPI ReservePay. The agentic payment protocol layer is being written right now by Pine Labs P3P and Google AP2, with Visa and Mastercard running parallel agentic-token programs. Building any of these in 2026 is a guaranteed commodity with negative margin and no distribution.

One proposed capability deserves immediate burial rather than audit: reading OTPs from the user's messages to complete payments. That architecture is not a moat; it is a liability with a clock on it. It exposes a live authentication secret to an LLM context that cannot guarantee deletion, it collides with the regulatory logic behind Additional Factor of Authentication, and it is trivially replicable by any competitor with SMS permissions. Section 3 kills it formally and identifies what actually replaces it: mandates, reserved funds, and delegated authorization instruments that already exist on Indian rails.

The verdict in one sentence

KIRANA's only defensible build is the Agent Financial Control Plane: the integrated, provable system that converts a user's signed delegation into a cryptographically bound, policy-checked, auditable transaction - operated for one vertical first, on rails KIRANA never owns.

What survives the kill-list is narrow and unglamorous, and that is precisely why it is defensible. Seven components survive: the verifiable delegation chain, the deterministic policy gate, cryptographic transaction binding, agent identity and key custody, agentic risk signals, reconciliation-of-unknown, and the hash-chained proof bundle that makes every agent-authorized rupee disputable after the fact. No single component is deep magic. P3P binds payments inside its own protocol; AP2 sketches mandates conceptually; UPI Circle delegates between humans; OPA and Cedar evaluate generic policies. The non-commoditized asset is the composition - a complete, evidence-producing control plane for machine-initiated spending, which today no PSP, network, or platform ships as a product a startup can rent. Section 4 examines each surviving component against the prior art and states, honestly, the condition under which each one collapses.

This document freezes the architecture so that implementation can start without re-litigating design. The freeze decisions are listed in Table 1.1 and elaborated through Chapter 5. Three of them are strategic rather than technical: KIRANA does not invent a public protocol, does not touch funds flow, and does not allow a single rupee of real money to move until the red-team suite passes in sandbox. Everything else - services, schemas, APIs, state machines, key custody, threat controls, a 90-day execution plan with acceptance gates, and the regulatory boundary map - is specified in the chapters that follow. Where a claim depends on a fact

the research corpus flagged as unverified (P3P startup access, ReservePay onboarding, AP2 production maturity), this document says so explicitly rather than guessing.

ID	Decision	Rationale in one line
D1	Kill OTP-to-agent permanently; never automate OTP, PIN, or any AFA secret	Secret exposure is un-deletable and regulatorily fragile; zero moat
D2	Do not invent a public protocol; KAP is an internal adapter contract only	P3P and AP2 already own the protocol layer; a startup protocol has no adopters
D3	Build the control plane; buy execution (PSP, rails, vaults) as adapters	Execution is certified commodity; verification and delegation are the wedge
D4	Freeze stack: TypeScript/Node modulith, Postgres 16, Redis 7, cloud KMS (Ed25519)	One language on one write path; extraction seams defined, microservices deferred
D5	The LLM never decides authorization; CAN_PAY() is deterministic code	Prompt injection and hallucination are existential in payments; code fails closed
D6	No real money until Gate G3: 19-attack red-team suite green in sandbox	Agent-compromise blast radius must be proven zero-cost before live rails
D7	Operate as a technology service provider; never aggregate or touch funds flow	Avoids PA/TPAP licensing gravity; partners move money, KIRANA proves authority
D8	Position: Agent Financial Control Plane, grocery-first in India; universal ambitions deferred	Vertical trust, merchant integrations, and rail fluency beat thin protocol layers

Table 1.1 - Freeze decisions (binding unless re-opened by the Architecture Review Board)

2. Evidence Base and Method

This blueprint is derived from the KIRANA master research corpus - the sixty-part research program covering the agentic payments landscape (P3P, AP2, UPI AutoPay / ReservePay / Circle, Razorpay, Visa and Mastercard agentic programs, FIDO, PCI DSS, and the RBI / NPCI regulatory frame) - and from engineering judgment applied to that corpus. It does not invent facts to fill gaps. Wherever the underlying research classified a claim as marketing rather than documented capability, that classification is preserved here, and the claim is either excluded or marked for verification. This discipline matters more in payments than elsewhere: an architecture that assumes an API which does not exist, or a regulatory permission no one has granted, is not an architecture - it is a wish with diagrams.

Every material claim in this document carries one of five evidence grades, following the classification scheme fixed in the research brief. The grades are used in the tables throughout: [OFFICIAL] for primary documentation from NPCI, RBI, PCI SSC, EMVCo, FIDO, or a named vendor's official developer documentation; [STANDARD] for published protocol specifications such as OAuth 2.0, DPoP, WebAuthn, or JOSE; [COMMERCIAL] for vendor-marketed capabilities that are plausibly real but whose startup-access terms are unconfirmed; [ENGINEERING INFERENCE] for design conclusions drawn here from

established primitives - the majority of the frozen design; and [UNVERIFIED] for claims that must be confirmed before capital is committed. No claim in this document is presented at a higher grade than its source supports.

#	Claim the architecture leans on	Grade	Verification path
V1	Pine Labs P3P is production-available and onboardable by an early-stage startup on commercial terms	[COMMERCIAL]	Direct merchant/developer onboarding conversation; obtain rate card, SLA, and sandbox access in writing
V2	UPI ReservePay (reserved funds / block-and-pay) is accessible to a TPAP-integrated startup, not only pilot banks	[UNVERIFIED]	NPCI circulars plus PSP partner confirmation of live ReservePay handle provisioning
V3	Google AP2 IntentMandate / PaymentMandate is implementable today beyond reference code	[COMMERCIAL]	AP2 spec and sample repos; confirm no ecosystem gate blocks independent mandate issuance
V4	Razorpay UPI AutoPay e-mandate creation and webhook reconciliation behave as documented at startup volume	[OFFICIAL]	Sandbox + one live low-value mandate end-to-end during Phase 2
V5	RBI payments data localization is satisfiable with the chosen cloud region topology	[OFFICIAL]	Read the storage circular text against the selected region architecture with counsel

Table 2.1 - Load-bearing facts that require verification before build capital is committed

Two consequences follow from this honesty, and both are frozen into the plan. First, the 90-day execution plan (Chapter 10) front-loads the cheapest possible verification of V1 through V4 into Phases 0 and 2, so that the architecture is falsified early against reality rather than late against production traffic. Second, every provider-facing component is wrapped behind the provider abstraction of Chapter 5, so that if P3P access fails to materialize, the control plane survives unchanged while the adapter slot is filled by Razorpay mandates alone. The design goal is that no single unverified external dependency can invalidate the control plane itself, which is the part KIRANA actually owns.

3. The Commoditization Kill-List

This chapter exists to prevent the most expensive mistake available to this project: rebuilding what the ecosystem already sells. Each row of Table 3.1 names a capability the original KIRANA concept could plausibly be talked into building, names who already owns it, and issues a verdict. The verdicts are not aesthetic preferences; they follow from three tests. Does a certified, priced, startup-accessible supplier exist? Would building it require licenses or network membership a startup cannot obtain in under a year? And does owning it generate switching costs, or merely parity? Anything that fails the tests belongs to the kill-list, no matter how central it feels to the original pitch.

Capability	Who already owns it	Verdict
Payment rails (UPI, card schemes)	NPCI; Visa / Mastercard	DO NOT BUILD - ever. Network membership is not purchasable by a startup
PSP / gateway / orchestration	Razorpay, Cashfree, Juspay, Pine Labs	BUY / INTEGRATE as adapters behind the broker abstraction
Recurring mandate infrastructure	UPI AutoPay e-mandates via PSP (NPCI)	USE as-is; the delegated-spend semantics belong to the control plane
Reserved-funds / block-and-pay	UPI ReservePay (NPCI, bank-gated pilots)	INTEGRATE when access lands (V2); never emulate holds on other rails
Agentic payment protocol	P3P (Pine Labs, shipping); AP2 (Google + partners, spec-stage)	ADOPT concepts and interfaces; never publish a competing public protocol
Credential vaulting / tokenization	Network tokens; PSP card vaults under PCI	NEVER STORE raw credentials; stay out of PCI scope by construction
OTP-to-agent automation	Nobody legitimate - and that is the signal	KILL. Architecturally dead; see 3.1
Own TEE / HSM fleet	Cloud KMS, HSM, confidential-compute vendors	BUY managed; own fleet is capex without differentiation
Generic agent runtime / LLM orchestration	Model vendors and agent frameworks, improving monthly	COMMODITY INPUT - differentiate above it, never inside it
Chat, search, and cart UX	Every marketplace and assistant	TABLE STAKES - required, but earns zero switching cost

Table 3.1 - The kill-list: what KIRANA must not build

3.1 Why OTP-to-agent is dead, formally

The original product idea - the agent reads a payment OTP from the user's SMS and completes the transaction, deleting the code afterwards - fails on three independent grounds, any one of which is sufficient. Ground one: exposure is permanent in practice. Once an OTP enters the agent's process it exists in heap, logs, crash dumps, tracing spans, and the model provider's context infrastructure; deletion after the fact is secret hygiene theater, and the research corpus correctly distinguishes secret minimization from impossible secret deletion. Ground two: authorization logic is wrong. An OTP authenticates a session at a moment in time; it does not express amount, merchant, category, expiry, or revocation. A stolen or replayed OTP is unconstrained authority, which is exactly the property the whole delegated-authorization thesis exists to eliminate. Ground three: it is regulatorily and commercially fragile. India's AFA regime treats that second factor as the customer's consent; an agent consuming it silently converts a security control into a UX feature, and any PSP risk team that notices will terminate the integration. The replacement is not a better way to swallow OTPs; it is architectures that never need them: UPI AutoPay mandates for recurring authority, ReservePay for reserved-funds high-ticket flows, and the signed delegation instruments this blueprint specifies.

If a capability can be summarized as 'the AI reads the customer's second factor', it is not a product. It is an incident report with a launch date.

3.2 Why KIRANA does not publish a protocol

The research program asks whether to design a public Kirana Agent Payment Protocol (KAP). The skeptical answer is no - publishing protocols is a winner-take-most game that requires adopters, and a seed-stage startup has none. P3P is in market with an implemented agentic flow; AP2 has platform-scale distribution behind its mandate model; Visa and Mastercard are wiring agentic tokens into their networks. A fourth protocol from a company with no merchants and no wallets would add fragmentation for everyone else and zero defensibility for KIRANA. What survives of KAP is real but internal: the message shapes of Chapter 8 (AgentAuthorization, PaymentIntent, PaymentAuthorization, PaymentResult, PaymentReceipt, RevocationEvent) become KIRANA's internal adapter contract - the stable interface its own providers, agents, and clients code against. If the control plane earns distribution, these shapes can later be proposed outward from a position of strength; protocol publishing is a reward for adoption, never a strategy for getting it.

The uncomfortable conclusion of this chapter is also its most useful finding: everything on the kill-list is exactly where the glamour sits - rails, protocols, vaults, the OTP trick. What remains after the kill-list is the unglamorous control-plane work that no platform has productized, because it is vertical-specific, liability-heavy, operationally grinding, and worth little until it is complete. That asymmetry - glamorous to skip, valuable when finished - is the actual shape of the opportunity, and the rest of this blueprint is the engineering plan for finishing it.

4. What Survives: The Defensible Wedge

Seven components survive the kill-list. For each one this chapter states what it is, checks it against the closest prior art the research corpus identified, and then - in the spirit of a skeptical review rather than a pitch - states the specific condition under which the component collapses into someone else's commodity. The claim defended here is deliberately modest: no component is individually deep tech, and the moat is the composition plus the operating data, not any single primitive. A competitor can buy every ingredient; what they cannot buy off the shelf is the complete, evidence-producing machine operated at transaction volume.

W1 - The verifiable delegation chain

A delegation is a signed, versioned artifact in which the user grants a specific agent a bounded spend authority: purpose, merchant scope, category, per-transaction and velocity limits, price-drift tolerance, substitution policy, confirmation thresholds, and expiry. The user signs it with a hardware-protected key at creation (passkey / WebAuthn ceremony); the agent later signs each transaction under it. Authority therefore

becomes an inspectable object rather than an assumed relationship. Closest prior art: AP2's IntentMandate and PaymentMandate define this shape conceptually but are not an issued, revocable credential on Indian rails; UPI Circle delegates between people with fixed sub-limits, not between a person and an agent under rich policy; OAuth scopes are stringly-typed and not transaction-bound. Collapse condition: if a wallet-scale platform issues AP2-native mandates in India with revocation and policy richness, W1 becomes integration work. Mitigation: keep the artifact schema portable so KIRANA can consume external mandates as just another delegation source.

W2 - The deterministic policy gate

CAN_PAY() is a pure function from (delegation, proposal, ledger state, risk signal) to {ALLOW, CHALLENGE, DENY} plus a machine-readable reason. It is evaluated in a fixed order of twelve checks (Chapter 6), fails closed, and never consults the model. Closest prior art: OPA/Rego and AWS Cedar are generic engines; banks have static spent-limit engines; none ship agent-spend semantics such as price drift versus signed intent, substitution policy, cart-hash binding, or confirmation thresholds expressed against delegated authority. Collapse condition: policy semantics are configuration, not cryptography - copyable in weeks by anyone who has seen this document. The durable part is the rule library tuned by real abuse and false-positive data, which is why Chapter 10 forces early sandbox exposure: the gate that has seen ten thousand attempted agent spends is a different asset from the one described in a document.

W3 - Cryptographic transaction binding

Every authorization is bound to exactly one transaction: cart hash (canonicalized JSON, SHA-256), amount, currency, merchant identity, agent key, and a single-use nonce, signed by the agent key and verified by the gate. The executed payment must match the bound tuple; amount drift beyond tolerance is a DENY, not a warning. Closest prior art: P3P performs resource and amount binding inside its protocol - when it is usable, KIRANA consumes that binding through the P3P adapter rather than reproducing it; outside P3P, no cross-provider binding abstraction exists for UPI flows. Collapse condition: the primitives (signatures, hashes, nonces) are commodities; if a rail-native binding standard emerges and is universally adopted, W3 reduces to adopting it. Until then, the binding schema and its evidence chain are the asset.

W4 - Agent identity and key custody

Each agent instance holds a distinct Ed25519 key pair generated inside a cloud KMS (private key non-exportable); delegations name the agent public key, so authority is bound to a verifiable identity rather than to a bearer token that can be lifted. Broker APIs accept DPOP-style proofs so tokens are unusable when stolen. Closest prior art: FIDO's agentic authentication work and WebAuthn give the user-side ceremony today; DPOP and mTLS are standardized. Collapse condition: partial and welcome - if FIDO standardizes agent-attested keys, KIRANA adopts them for the user side while retaining its custody model for the agent side. Identity is infrastructure, and KIRANA's version must remain swappable by design.

W5 - Agentic risk signals

Machine-initiated spending has a different risk topology from human spending: authorization age versus first spend, velocity patterns that look like retry storms, cart mutations between bound and executed states, merchant-category drift within one delegation, and content-shape signals suggestive of injected instructions. The v0 risk engine scores these into LOW / MEDIUM / HIGH with deterministic outcomes (allow / challenge / block). Closest prior art: PSP risk engines score card transactions, not delegated-agent behavior; they cannot see delegation age, cart-hash deltas, or prompt-injection suspicion. Collapse condition: this is the most data-moat-like component - it improves with volume - but it is also the least finished at day zero, which is why v0 ships with three signals and honest thresholds, not a neural mythology.

W6 - Reconciliation of unknown

Agentic flows retry; retries collide with timeouts; and the most dangerous state in payments - EXECUTING with no definitive outcome - becomes common rather than exceptional. KIRANA treats UNKNOWN as a first-class state with a worker-owned promotion path to SETTLED or DROPPED (Chapter 8, Figure 8.2): idempotency keys collapse duplicate executions, provider inquiries and signed webhooks are cross-checked, and no order is ever placed from an ambiguous state. Closest prior art: every serious PSP has internal reconciliation; none exposes it as a product for agent-initiated flows, and the agent-orchestration frameworks pretend the problem does not exist. Collapse condition: low - this is operational craft, unglamorous and unpublishable, which is exactly what makes it durable.

W7 - The proof bundle

Every transaction accumulates a hash-chained evidence record: the signed delegation version, the typed proposal, the cart hash, the CAN_PAY() decision with its rule evaluations, the binding signature, the risk verdict, the provider references, and the settlement proof. The bundle is what lets a human dispute a machine spend in front of a bank, a regulator, or a court - and it is the direct answer to the liability question every partner will ask. Closest prior art: PSPs keep their own logs; AP2's mandate model implies auditability but ships no evidence product; no one assembles the full chain from user intent to settlement into a verifiable artifact. Collapse condition: the bundle is cheap to copy but worthless without the system that generates it truthfully; its value compounds with W5's data and with partner trust.

The wedge, stated honestly

KIRANA's defensible asset is not any single primitive - it is the operated composition of W1-W7 producing a proof bundle per transaction, plus the abuse data that only live volume can generate. Every ingredient is purchasable; the machine is not.

5. Frozen Architecture

The architecture is frozen as a three-zone system with hard trust boundaries. Zone 0 is untrusted reasoning: the LLM runtime, connectors, and anything that touches unbounded text. Zone 1 is the control plane: deterministic services that own every authorization decision, every secret-adjacent operation, and every audit record. Zone 2 is execution: certified third parties (PSPs, rails, merchants) that move money through adapters. The controlling idea is a strict diet for the model - the LLM may understand, search, compare, and propose, but the only thing it can emit toward money is a typed PaymentIntentProposal that enters Zone 1 through one door, and the only thing it receives back is a decision and a result. Figure 5.1 shows the frozen system; Figure 5.2 shows what may and may not cross the boundary; Figure 5.3 shows the end-to-end payment lifecycle.

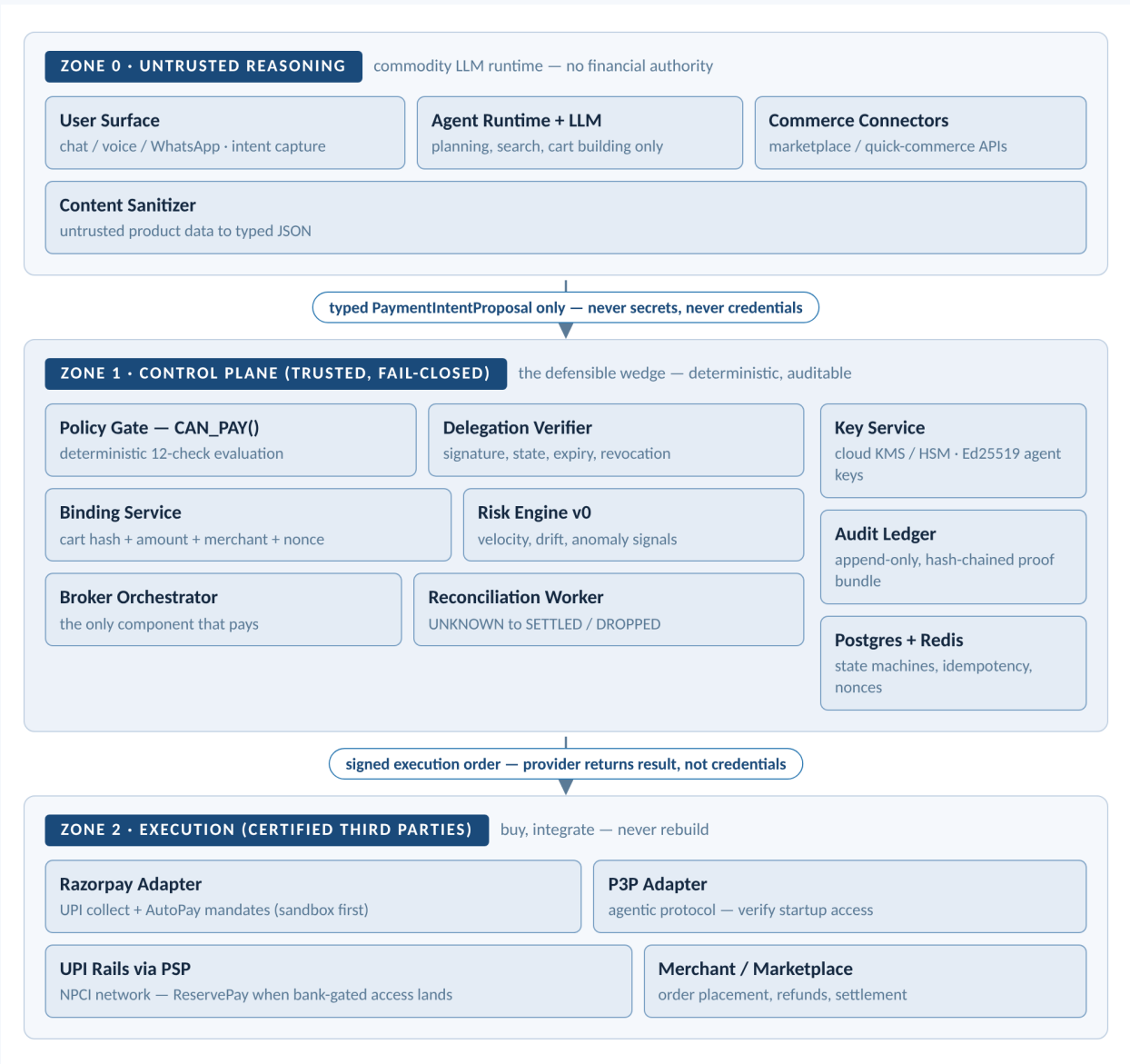


Figure 5.1 - Frozen architecture: three zones, hard boundaries, one payment door

5.1 Trust boundary rules

Six rules define the boundaries, and every one of them is enforced in code, not in documentation. B1: the LLM emits only typed proposals validated against the Chapter 8 schema; malformed input is rejected at the door. B2: only the Broker Orchestrator executes financial calls; no other component holds provider credentials. B3: secrets (UPI PIN, CVV, PAN, passwords, private keys, AFA codes) never enter Zone 0 - not in prompts, not in logs, not in errors, not in traces. B4: every Zone 1 decision is an audit event appended to the hash chain before any side effect occurs. B5: the system fails closed - timeouts, unknown states, schema drift, and provider inconsistencies all resolve to DENY or to reconciliation, never to allowance. B6: revocation propagates to the Policy Gate in at most five seconds, evaluated server-side at decision time; the agent's local view is never trusted on freshness.

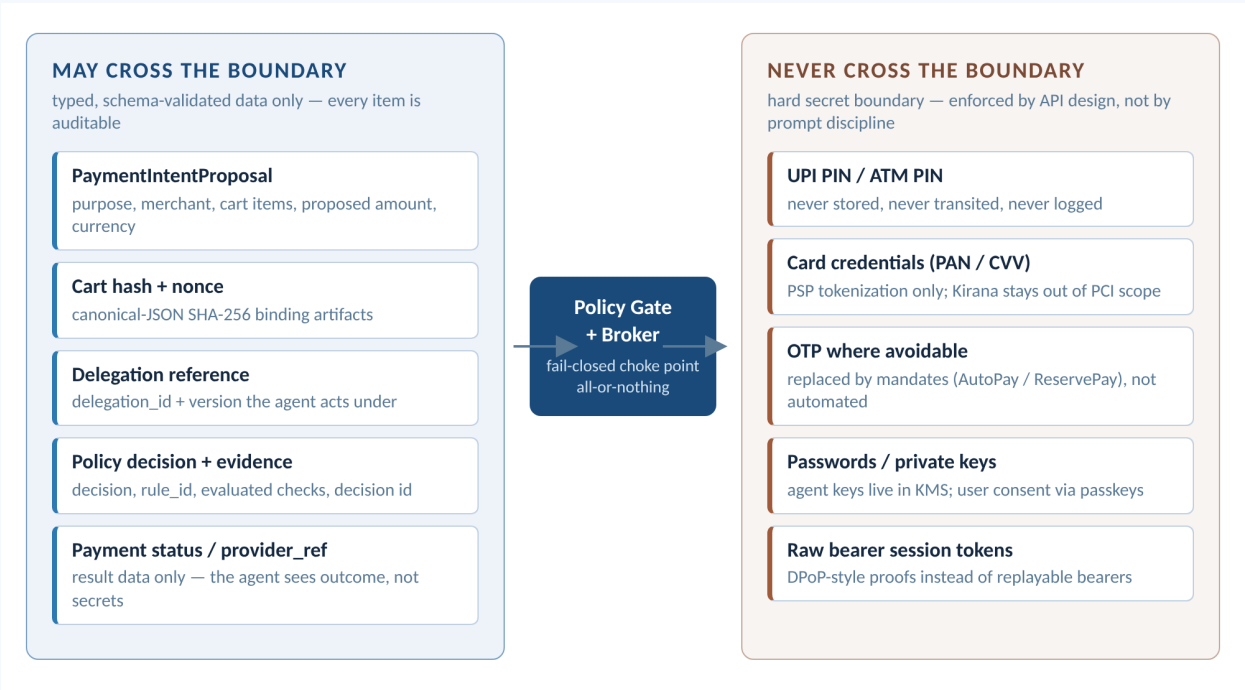


Figure 5.2 - The secret boundary: what may cross, and what must never cross

5.2 Component responsibilities

Component	Owns	Interface (abbreviated)
Delegation Service	Delegation lifecycle, versions, user signing ceremony records	createDelegate, suspend, resume, revoke, getActive
Policy Gate	CAN_PAY() evaluation, check order, decision records	evaluate(delegation, proposal, ledger, risk) to Decision
Binding Service	Cart canonicalization, hash, nonce registry, binding signatures	bind(proposal) to BoundTransaction; consumeNonce

Component	Owns	Interface (abbreviated)
Risk Engine v0	Velocity, drift, anomaly scoring; verdict records	score(bound, context) to {LOW, MEDIUM, HIGH}
Broker Orchestrator	Provider selection, execution, idempotency keys, webhooks	execute(bound, decision) to PaymentResult
Reconciliation Worker	UNKNOWN promotion, provider inquiry, webhook backlog	reconcile(intentId) to {SETTLED, DROPPED}
Key Service	Agent Ed25519 keys in KMS, rotation, proof verification	sign(agentId, payload), verifyProof(proof)
Audit Ledger	Append-only hash chain; proof bundle assembly	append(event), chain(verifier), bundle(intentId)
Provider Adapters	Capability discovery, API translation, sandbox toggles	capabilities(), execute(), status(), refund()
Connector Sanitizer	Untrusted commerce content to typed JSON	parseProduct(payload) to typed DTO (reject on injection shape)

Table 5.1 - Zone 1 component specification (state owners are authoritative for their state machines)

5.3 Pinned stack and the monolith-first decision

Concern	Frozen choice	Reason in one line
Language / runtime	TypeScript on Node 20 LTS, ESM	One language across control plane, adapters, and agent tooling; typed contracts end to end
Service shape	Modulith: one deployable, hard internal package boundaries	The control plane is one write path; premature microservices multiply failure modes, not safety
Primary store	Postgres 16 + Prisma	Transactional state machines, unique constraints for idempotency, JSONB for artifacts
Hot state	Redis 7	Nonce registry, revocation cache (5 s staleness cap), idempotency locks
Keys / crypto	Cloud KMS with HSM backing; Ed25519; DPoP-style proofs	Non-exportable agent keys without owning hardware
Async work	Transactional outbox + workers (no external workflow engine at MVP)	Fewer moving parts; outbox gives exactly-once side effects with at-least-once delivery
Observability	OpenTelemetry traces + metrics; structured logs with secret redaction at the sink	Every decision traceable to its inputs without leaking secrets
Extraction seams	Broker, Reconciliation, Risk pre-identified for future service extraction	Boundaries drawn now along the lines future load will demand

Table 5.2 - Frozen technology choices

The monolith-first decision deserves its skeptical note: nothing in this design depends on the monolith being permanent. The seams chosen for later extraction (Broker, Reconciliation, Risk) are precisely the components with independent scaling and compliance futures, and the internal package boundaries are enforced by tooling so that extraction is a mechanical act rather than a rescue. What the freeze does forbid is the opposite failure - starting as five microservices whose inter-service failure modes get designed in a hallway and discovered in production. In payments, the first version of a distributed system should be as non-distributed as the team can tolerate.

6. Delegation Model and Policy Engine

The delegation object is the unit of authority and the anchor of the whole control plane, so its schema is frozen here. It is created in a user ceremony (passkey / WebAuthn signing over the exact canonical bytes), stored immutably, versioned on every modification, and referenced by every downstream decision. Amounts are integer paise. The example below shows the default grocery delegation used across the research corpus: INR 500 per transaction, INR 1,500 per day, INR 15,000 per month, three named merchants, zero price drift, substitution requires asking, and anything above the per-transaction limit requires an explicit user confirmation challenge rather than an automatic decline.

Frozen delegation artifact (v1, abridged; signatures omitted for width)

```
{ "delegation_id": "dlg_01J9...c7", "version": 3, "principal": "user_123", "agent": {
  "id": "kirana_01", "key_id": "agpk_...", "pubkey": "ed25519:base64(...)" }, "purpose":
  "grocery_purchase", "merchant_scope": { "mode": "allowlist", "merchants": ["zepto",
    "blinkit", "bigbasket"] }, "category_scope": ["grocery"], "limits_paise": { "per_tx":
    50000, "daily": 150000, "weekly": 400000, "monthly": 1500000 }, "currency": "INR",
  "price_drift_bps": 0, "substitution": "ASK", "require_confirmation_above_paise": 50000,
  "valid_from": "2026-08-31T00:00:00Z", "expires_at": "2026-09-30T00:00:00Z", "nonce":
  "n_9f14...", "user_signature": { "alg": "Ed25519", "key": "webauthn:...", "over":
  "canonical_json(v1 fields)" }, "status": "ACTIVE" }
```

6.1 CAN_PAY(): the deterministic gate

The gate evaluates twelve checks in a fixed order and stops at the first failure, because ordering is itself a security decision: cheap cryptographic and state checks fail before expensive risk checks, and an attacker learns only the earliest failing rule. The function is pure - no network calls, no model calls - and every evaluation writes a decision record with the evidence of each check, which the Audit Ledger chains. CHALLENGE is a distinct outcome, not a soft ALLOW: it pauses execution behind a user confirmation step, and the confirmation itself becomes a signed event. The default posture for every unclassifiable condition is DENY.

#	Check	Fails with	Notes
1	AUTH_IDENTITY	DENY	Agent proof valid; key matches delegation.agent
2	DELEGATION_STATE	DENY	ACTIVE only; suspended, revoked, expired reject here
3	TIME_WINDOW	DENY	valid_from, expires_at; server clock is authoritative
4	MERCHANT_SCOPE	DENY	Allowlist or category rules on the bound merchant
5	CATEGORY_SCOPE	DENY	Merchant category within delegation.category_scope
6	PER_TX_LIMIT	DENY	amount + drift never above per_tx limit
7	VELOCITY	DENY	Daily / weekly / monthly sums from the ledger
8	PRICE_DRIFT	DENY	Checkout price versus signed proposal within drift_bps
9	CART_BINDING	DENY	Fresh cart hash equals bound hash; no mutation mid-flight
10	CONFIRMATION	CHALLENGE	Above threshold: requires a fresh signed user yes
11	RISK_GATE	DENY / CHALLENGE	HIGH denies; MEDIUM challenges; LOW proceeds
12	NONCE_FRESH	DENY	Single-use nonce, 60 s replay window, Redis-backed

Table 6.1 - CAN_PAY() evaluation order (fixed, fail-closed, first failure wins)

Two worked examples fix the semantics. A proposal for INR 427 at a named grocery merchant under an ACTIVE delegation with clean velocity, zero drift, matching cart hash, LOW risk, and a fresh nonce is ALLOWed end to end. An attempt to push the same cart through at INR 4,270 against an unknown electronics merchant fails at check 4 (MERCHANT_SCOPE) before the attacker ever learns whether the velocity budget would have permitted it - and the failed attempt is itself an audit event that feeds check 11 next time. The gate never explains more than the earliest failing rule; that is deliberate information discipline.

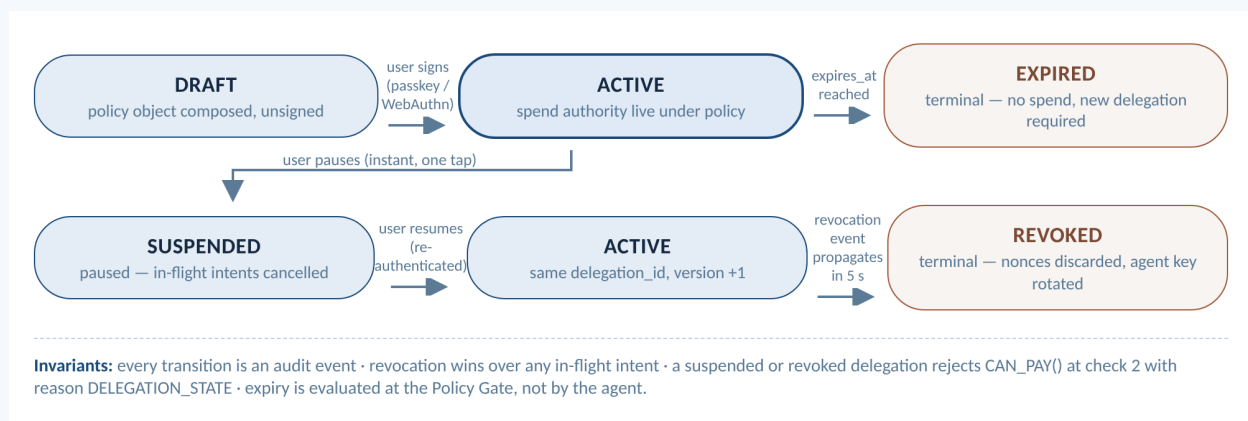


Figure 6.1 - Delegation lifecycle: every transition signed, versioned, and audit-chained

7. Data Model

The schema is organized around one rule: money-adjacent state transitions live in tables with unique constraints that make double-spending representable but not committable. Idempotency is enforced at the database layer first (unique keys), at Redis second (locks), and by provider-side idempotency keys third. All money is integer paise; all timestamps are UTC; all artifacts that users or agents sign are stored with their canonical bytes so signatures remain verifiable for the retention period. Tables below are specified to the level a migration can be written from; column types follow obvious Postgres mappings (JSONB for artifacts, BIGINT for paise).

Table	Key columns	Constraints and indexes
users	id PK, phone_hash, passkey_ids jsonb, created_at	unique(phone_hash); soft-delete flag
agents	id PK, owner_user_id FK, kind, status, created_at	index(owner_user_id); status in (ACTIVE, RETIRED)
agent_keys	id PK, agent_id FK, kms_key_ref, pubkey, alg, rotated_at, revoked_at	unique(pubkey); index(agent_id, revoked_at)
delegations	id PK, user_id FK, agent_id FK, artifact jsonb, canon_bytes bytea, version, status, valid_from, expires_at	unique(user_id, agent_id, version); index(status, expires_at)
revocations	id PK, delegation_id FK, reason, requested_by, propagated_at	index(delegation_id); feeds Redis cache invalidation

Table 7.1 - Core authority tables

Table	Key columns	Constraints and indexes
payment_intents	id PK, delegation_id FK, proposal jsonb, cart_hash, status, created_by	unique(delegation_id, cart_hash) where status active; index(status)
authorization_decisions	id PK, intent_id FK, can_pay jsonb, rule_fired, risk jsonb, decided_at	index(intent_id); immutable, append-only
payment_transactions	id PK, intent_id FK, bound_artifact jsonb, broker_idem_key, provider, provider_ref, amount_paise, status	unique(broker_idem_key); unique(provider, provider_ref); index(status)
payment_events	id PK, tx_id FK, type, payload jsonb, source (webhook, poll, worker), received_at	unique(source, provider_event_id) for webhook dedup; index(tx_id)
reconciliation_jobs	id PK, tx_id FK, attempts, next_run_at, outcome, evidence jsonb	index(next_run_at) where outcome is pending
refunds	id PK, tx_id FK, provider_ref, amount_paise, status, reason	unique(provider_ref); index(tx_id)

Table 7.2 - Transaction and decision tables

Table	Key columns	Constraints and indexes
audit_events	id PK, seq, at, actor, type, payload jsonb, prev_hash, entry_hash	unique(seq); entry_hash = SHA-256(prev_hash, canonical(payload)); chain walk job verifies hourly
idempotency_keys	key PK, scope, request_hash, locked_at, released_at	unique(key, scope); TTL reaper
nonces	nonce PK, delegation_id FK, intent_id, expires_at	unique(nonce); Redis primary, Postgres durable mirror

Table 7.3 - Integrity and idempotency tables

Retention and encryption follow the frozen posture: signed artifacts and decision records are retained for the regulatory horizon with encrypted-at-rest storage and India-region residency per the localization constraints of Chapter 11; nonce and idempotency data are short-lived by design; nothing in any table is ever a raw card credential, because Zone 1 never receives one. The audit chain is the one table optimized for distrust rather than speed - hourly chain-walk verification runs as a worker, and any broken link pages a human before anything else in the system is allowed to forget the event.

8. API Contracts and State Machines

The internal API surface (the KAP-shaped adapter contract frozen by decision D2) is deliberately small: authority in, proposal in, decision out, execution invisible behind the broker, and a single emergency control. All endpoints require a DPoP-style proof bound to the caller's agent key; all mutating endpoints require an Idempotency-Key header; all responses share one error envelope; and every endpoint declares its audit requirement in the contract itself. Rate limits are per agent identity, not per token, because tokens are stealable and identities in KMS are not.

Endpoint	Method	Auth / proof	Idempotent	Notes
/v1/delegations	POST	User WebAuthn ceremony + agent enrollment	Yes (client token)	Creates ACTIVE delegation; returns signed artifact
/v1/delegations/:id	GET	Agent proof or user session	Read	Returns current version + status
/v1/delegations/:id/suspend	POST	User session	Yes	Pause; in-flight intents cancelled
/v1/delegations/:id/resume	POST	User session (re-auth)	Yes	Version +1
/v1/delegations/:id/revoke	POST	User session	Yes	Terminal; propagates in 5 s
/v1/payment-intents	POST	Agent DPoP proof	Yes (Idempotency-Key)	Runs CAN_PAY(); returns decision + evidence

Endpoint	Method	Auth / proof	Idempotent	Notes
/v1/payment-intents/:id/authorize	POST	Agent DPOP proof	Yes	Binds nonce, signs transaction, enters execution
/v1/payments/:id	GET	Agent proof or user session	Read	Status; UNKNOWN surfaces as-is
/v1/payments/:id/reconcile	POST	Worker identity	Yes	Worker-only; promotes UNKNOWN
/v1/payments/:id/refund	POST	User session + policy	Yes	Broker-initiated provider refund
/v1/agent/capabilities	GET	Agent proof	Read	Provider capability discovery for the agent runtime
/v1/kill-switch	POST	User session	Yes	Global pause: all delegations of this user SUSPENDED at once

Table 8.1 - API surface (v1)

POST /v1/payment-intents - request and response (abridged)

```
REQ { "delegation_id": "dlg_...c7", "idempotency_key": "ik_...", "proposal": { "purpose":
"grocery_purchase", "merchant": "zepto", "category": "grocery", "cart": [ { "sku":
"potato-1kg", "qty": 1, "unit_paise": 3200 }, ... ], "amount_paise": 42700, "currency":
"INR" } } RESP { "decision": "ALLOW", "rule_fired": null, "intent_id": "pi_...89",
"expires_in_s": 60, "evidence": { "checks": "1-11 PASS", "risk": "LOW", "decision_id":
"dec_...f2" } }
```

8.1 Payment state machine and the meaning of UNKNOWN

The payment state machine has exactly one ambient rule: no financial action is ever triggered from an ambiguous state. Transitions are the ones frozen in Figure 8.1; the transition table adds the guards. SUCCESS requires a provider reference plus an amount and merchant match across two independent signals (signed webhook or poll), because payment success is not the same fact as order success, and the control plane never conflates them. FAILED is deterministic and terminal for that intent - the agent may build a new proposal, but the failed intent itself can never be retried, which is what makes retry storms structurally impossible rather than merely discouraged.

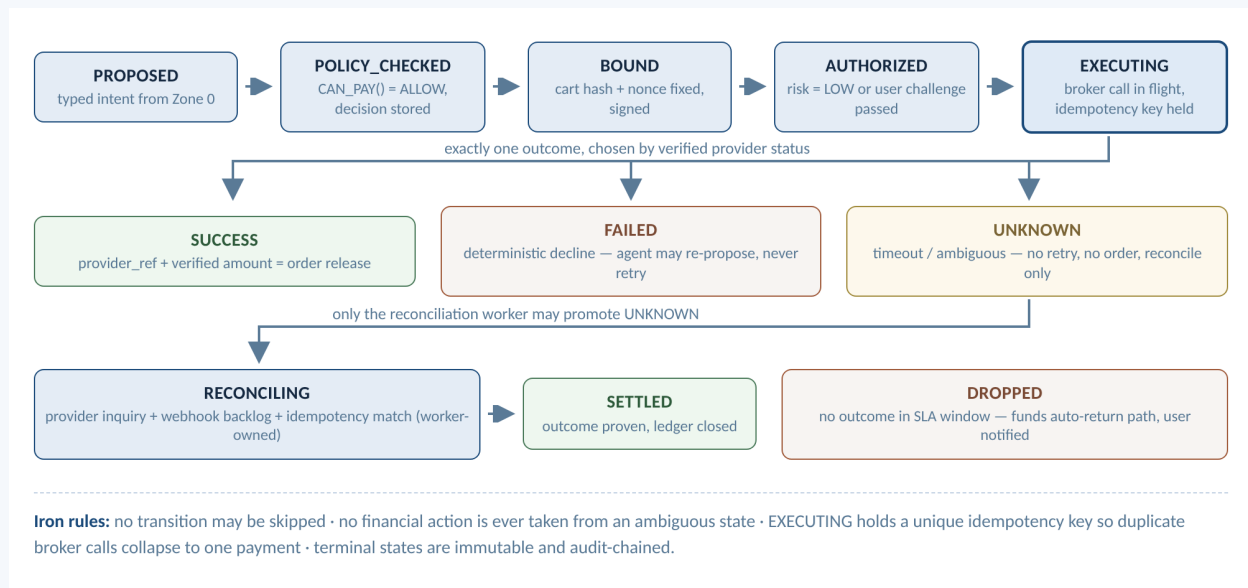


Figure 8.1 - Payment state machine: one happy path, three outcomes, worker-owned promotion

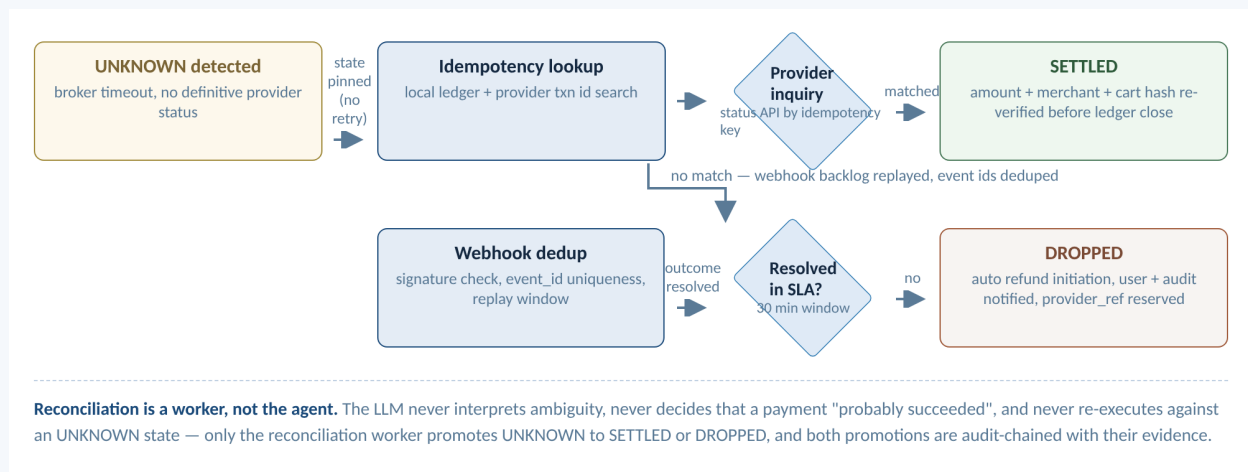


Figure 8.2 - UNKNOWN is normal: reconciliation is a worker with evidence, never an agent judgment call

The lifecycle that stitches delegation, policy, binding, execution, and receipt together across zones is shown once more end to end in Figure 8.3, in the exact order the Research Brief specified it: understand, propose, gate, verify, bind, risk, execute, verify, order, receipt. The sequence is the product. Every arrow in the figure corresponds to an API call in Table 8.1 and an audit event in Table 7.3, which is the property that lets an auditor replay any transaction from its proof bundle without trusting anyone's memory.

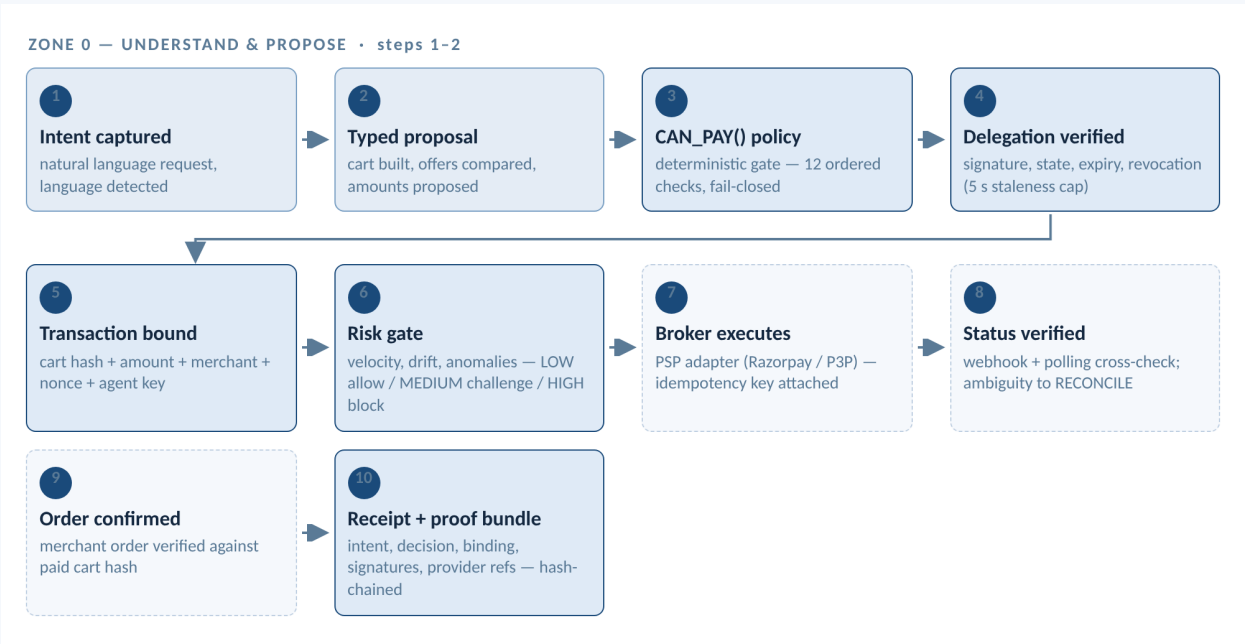


Figure 8.3 - The full agentic payment lifecycle (steps 1-10) across the three zones

9. Security Model and Threat Matrix

The security model is organized around one admission: the agent will eventually be compromised. Prompt injection is not a bug to be patched but an environmental condition of deploying LLMs against unbounded text - product listings, order statuses, and user messages are attacker-controlled surfaces. The design therefore assumes Zone 0 is hostile and concentrates all guarantees in Zone 1, where behavior is deterministic and every action is chained. Key custody follows: agent private keys are generated and used inside cloud KMS with HSM backing and are non-exportable; the user's consent artifacts are signed with passkeys under WebAuthn so the user's secret never exists on any server; and broker-facing tokens are DPoP-style proof-bound so a stolen bearer string is inert. Rotation is scheduled and revocation-propagated; compromise of one agent key invalidates exactly that agent's delegations, not the user's entire financial life.

9.1 Secret boundary

Class	Items	Rule
Never	UPI PIN, CVV, PAN, passwords, private keys, AFA / OTP codes, session secrets	Not in prompts, tools, logs, traces, errors, or screenshots; violation is a build-breaking test failure, not a policy note
Never-by-design	Raw card credentials of any form	Zone 1 never receives them; PSP tokenization keeps Kirana outside PCI scope
Allowed	Payment status, transaction and provider refs, amounts, currency, decision and evidence, order id	These are results, not authority; the agent narrates them, it cannot mint them

Table 9.1 - What the LLM may see, ever

9.2 Compromised-agent threat matrix

The matrix consolidates the red-team program of the research corpus into the attacks that matter against the frozen architecture. Each row names the attack, the control that stops it, and - with the honesty a review board requires - the residual risk that remains after the control. Anything whose residual risk was judged unacceptable was redesigned out during freezing; what remains here is the defended position, not a wish list.

Attack	Defense (frozen)	Residual risk
Spend above per-tx or velocity limits	Checks 6-7 at the gate; limits recomputed from the ledger at decision time	Ledger read skew within one transaction boundary; bounded by uniqueness constraints
Change merchant / category after delegation	Checks 4-5; bound merchant signed at authorize; PSP payload compared at execution	Merchant identity spoofing at connector level - mitigated by allowlisted API endpoints only
Amount manipulation / cart swap	Cart hash + drift check (8-9) + binding signature; provider call carries bound amount	Checkout-time price change within drift tolerance; drift = 0 bps by default so any change re-proposes
Replay a bound transaction	Single-use nonces (12), 60 s window, unique idempotency keys	None known within window; window exhaustion causes DENY, which is safe
Use expired or revoked delegation	Checks 2-3 against 5-second revocation cache at decision time	Sub-5-second race between revoke and an in-flight EXECUTING; mitigated by cancel-on-revoke of in-flight intents
Use another user's delegation	Delegation names agent pubkey; identity check 1 binds proofs to the delegation	Key compromise of an agent - blast radius limited to that agent's own delegations
Prompt injection via product content	Sanitizer parses untrusted text to typed DTOs; content never reaches instruction channel	Novel semantic injection that alters ranking (not authority) - gate still bound by policy
Duplicate payment via double submit	Idempotency-Key + unique(broker_idem_key) + provider idempotency	Provider-side idempotency gaps - covered by reconciliation cross-check
Blind retry on timeout	FAILED terminal; UNKNOWN blocks retry and forces reconcile	None; retry attempt is itself an alertable audit event
Webhook forgery / replay	Signature verification, unique(provider_event_id), timestamp window	Provider key compromise - out of scope, detected via cross-signal mismatch
Call PSP directly with stolen credentials	Provider credentials exist only in Zone 2 adapter scope; per-adapter, KMS-resident	Adapter host compromise - bounded by per-adapter scoping and anomaly alerts
Token theft (bearer capture)	DPoP-style proofs bind tokens to agent keys; stolen strings inert	None material

Attack	Defense (frozen)	Residual risk
Log / trace secret leakage	Redaction at the sink; secrets never cross B3; CI test asserts absence	Misconfiguration drift - covered by the checklists of Chapter 10
Hallucinated tool use by the LLM	Typed schemas; unknown tool names fail closed at the door	None; the model has no payment-capable tool other than propose

Table 9.2 - Threat matrix: attack, defense, residual risk

Design admission

The dangerous agent is the one that wants to pay. KIRANA's answer is not to make the model better behaved - it is to make obedience verifiable: every rupee moves only through a door that checks signatures, state, limits, and evidence in a fixed order, and every check it passes is written down.

10. 90-Day Execution Plan

The plan compresses the research corpus's ten-prototype progression into four phases with hard acceptance gates and pre-declared kill criteria. The controlling constraint is Gate G1: no real money moves until the red-team suite passes against sandbox rails. Each phase ends in a review against its gate; a failed gate stops the program or triggers redesign, not schedule pressure. The phases deliberately mirror the verification items of Table 2.1 so that the most load-bearing unknowns are retired earliest and cheapest.

Item	Content
Build	Monolith skeleton with hard package boundaries; delegation + payment state machines as pure transition functions; audit ledger with hash chain and chain-walk verifier; CI with secret-pattern tests
Gate G0	State machine property tests prove: no skip transitions, UNKNOWN never executes, every transition emits an audit event; chain verifier detects tampering in CI
Kill criteria	If the state machine cannot be made total (every input maps to a defined transition) in two weeks, redesign before anything else

Table 10.1 - Phase 0 (weeks 1-2): skeleton and invariants

Item	Content
Build	Delegation service + WebAuthn creation ceremony; policy gate with the twelve checks; mock provider adapter; proof-bundle writer; kill-switch endpoint
Gate G1a	100 percent of CAN_PAY() decision-table cases pass; a scripted compromised agent (19 attacks) scores zero successful spends against the mock provider
Kill criteria	If any attack in the suite cannot be deterministically stopped in mock, the design is reopened - not the test

Table 10.2 - Phase 1 (weeks 3-6): authority and policy

Item	Content
Build	Binding service (canonicalization, nonces, signatures); broker orchestrator with idempotency; Razorpay adapter against sandbox (AutoPay mandate create / execute / webhook); V1 and V4 verification concludes here
Gate G1b	End-to-end: signed delegation to sandbox payment to verified webhook to proof bundle; duplicate-submit and timeout-chaos drills produce exactly one payment or zero payments, never two
Kill criteria	If sandbox webhook reconciliation cannot be made deterministic, defer AutoPay and re-scope to collect-only flows

Table 10.3 - Phase 2 (weeks 7-10): binding and sandbox rails

Item	Content
Build	Reconciliation worker with SLA clock; risk engine v0 (velocity, drift, authorization-age); P3P access verification (V1) behind the adapter; full red-team suite wired into CI
Gate G2 / G3	G2: chaos drills (provider timeout, webhook storm, Redis loss) resolve without double spends and with audit completeness; G3: red-team suite green in sandbox - only then may a single real low-value transaction be run
Kill criteria	Any real-money loss scenario discovered post-G3 freezes production rollout pending redesign

Table 10.4 - Phase 3 (weeks 11-13): reconciliation, risk v0, red team

Staffing is deliberately small for this shape of work: two senior engineers on the control plane, one on adapters and verification, one part-time on security review, and one on the agent-side tooling. The 90-day output is not a product; it is a defensible machine proof - the evidence that the architecture stops a malicious version of itself - plus the verification dossiers (V1-V5) that turn the remaining strategic unknowns into facts. That is the correct unit of progress for a payments control plane, and it is the artifact a partner diligence process will actually read.

11. Regulatory Boundary Map

India's payments regulation determines which parts of this blueprint are software, which are partnerships, and which are forbidden. The map below classifies each architectural component into what a startup can ship alone, what requires a PSP relationship, what requires NPCI or bank access, and what requires license-aware structuring. It is engineering guidance anchored on the research corpus's regulatory analysis - not legal advice - and the three claims marked in Table 2.1 as unverified (V1, V2, V3) sit exactly on the partnership-dependent rows. The structuring principle frozen by decision D7 is to remain a technology service provider: KIRANA verifies and proves authority, while licensed partners execute money movement, which keeps the licensing surface small by construction.

Component / activity	Ship alone?	Partnership or license requirement
Control plane software, delegation, policy, binding, audit	Yes	None - this is KIRANA's product surface
Sandbox payments, mock and PSP test modes	Yes	PSP sandbox agreements (Razorpay test keys)
Production UPI collection for users	No	PSP / TPAP arrangement under NPCI framework; KYC / AML sits with regulated partners
UPI AutoPay e-mandate creation and management	No	PSP support of e-mandate APIs; notification and revocation rules per NPCI
ReservePay reserved-funds flows	No	NPCI-gated; bank / pilot access path - verify (V2) before roadmap commitment
Funds aggregation, settlement, wallets	Avoid entirely	PA / PPI territory - excluded by decision D7; never hold funds
Card credential handling	Never	PCI scope avoided by construction; PSP tokenization only
Payments data storage and residency	With design	RBI localization: India-region storage for payments data; confirm topology with counsel (V5)
AFA / OTP handling	Never automate	Mandates and reserved-funds replace OTP consumption; agent never touches AFA secrets
Agent liability and dispute evidence	With design	Proof bundles position KIRANA as evidence provider; dispute liability stays with regulated actors - structure with counsel

Table 11.1 - Component-level regulatory boundary

The strategic reading of this map matters as much as the rows. The boundary is not an obstacle to the moat - it is the moat's shape. Because licensed execution is table stakes and universally available, and because the unlicensed control plane is precisely where nobody else has productized depth, the regulatory perimeter pushes KIRANA's effort toward the only place where differentiation is possible. The map also explains the partnership sequence: PSP first (immediate), P3P second when terms allow (V1), ReservePay third when bank-gated access opens (V2), and protocol participation last, from a position of demonstrated operation rather than aspiration.

12. Risk Register and Final Recommendation

#	Risk	Sev.	Mitigation (frozen)
R1	Platform absorption: a wallet-scale player ships AP2-native delegated payments in India	HIGH	Stay consumable: portable artifacts, adapter-ready design; compete on vertical depth, proof bundle, and merchant integrations the platform will not hand-hold
R2	P3P / ReservePay access never materializes for startups (V1, V2)	HIGH	Architecture runs on Razorpay mandates alone; P3P and ReservePay are adapter slots, not load-bearing walls
R3	Liability gap: an agent-authorized purchase goes wrong and no framework assigns blame	HIGH	Proof bundle per transaction; partner contracts assign liability by evidence; positioning as verification layer, not money mover (D7)
R4	Prompt injection evolves faster than sanitizer patterns	MED	Guarantees never depend on the sanitizer; authority path is deterministic regardless of content quality; injection signals feed W5
R5	Retry storms or provider flakiness cause duplicate or lost payments	MED	Idempotency triad (DB, Redis, provider), UNKNOWN-first semantics, reconciliation worker with SLA clock
R6	Agent key or KMS compromise	MED	Non-exportable keys, per-agent blast radius, scheduled rotation, revocation propagation in 5 s
R7	Regulatory drift (localization, AFA, agentic payments rules)	MED	TSP posture minimizes licensed surface; quarterly counsel review; adapter layer isolates rail changes
R8	Unit economics of low-ticket grocery cannot carry infrastructure cost	MED	Control plane is rail-agnostic; the same machine monetizes beyond grocery (subscriptions, B2B procurement) once proven

Table 12.1 - Top risks, severity, and mitigation

12.1 Final recommendation

Proceed - but proceed on the frozen scope. Build the control plane exactly as specified, buy every certified commodity behind adapters, verify the three load-bearing unknowns in the first half of the plan, and refuse every tempting detour toward rails, protocols, vaults, or OTP tricks that this document has killed with reasons attached. The recommendation is not 'build an AI grocery agent'; assistants are commoditizing underneath you while you read this. The recommendation is to own verification and proof for machine-initiated spending in a market whose rails are world-class and whose trust layer for agents does not exist yet - and to earn the right to broaden scope only after the machine has survived its own red team in sandbox, gate by gate, as specified in Chapter 10.

12.2 First actions on approval

The plan begins with unglamorous verification, because that is where the architecture is most falsifiable for the least money. The first week is organizational, not technical: freeze this document as the single design source of truth, open the three external verification tracks of Table 2.1, and stand up the Phase 0 skeleton with its state-machine property tests. Nothing in the first two weeks requires a partner conversation to have concluded, which is exactly why the plan starts there.

1. Adopt this blueprint as frozen v1.0; changes go through the Architecture Review Board with a written rationale, not hallway consensus.
2. Open the P3P onboarding conversation and the ReservePay access inquiry in parallel (V1, V2), in writing, so the answers become artifacts rather than memories.
3. Contract or designate counsel for the RBI localization read (V5) and the TSP structuring opinion behind decision D7.
4. Stand up Phase 0: monolith skeleton, delegation and payment state machines as pure functions, the hash-chained audit ledger, and the CI secret-pattern tests.
5. Staff the four-seat build team of Chapter 10 and schedule the Gate G0 review for the end of week two.

12.3 KIRANA's single most defensible technical idea

The per-transaction Proof Bundle: a signed, hash-chained evidence chain that binds user intent to delegation version, policy decision, transaction binding, risk verdict, and settlement outcome - one verifiable artifact per rupee an agent spends. Payment providers structurally cannot own it, because each sees only its own slice of the story; platforms will not own it, because it is vertical, liability-heavy, and operationally grinding; and every regulator, bank, and insurer that has ever had to adjudicate a disputed machine payment will recognize its value on sight. KIRANA's product is not moving money - it is making delegated money provable, and that is the layer the entire agentic payments stack is missing.